```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('/content/placement.csv')
```

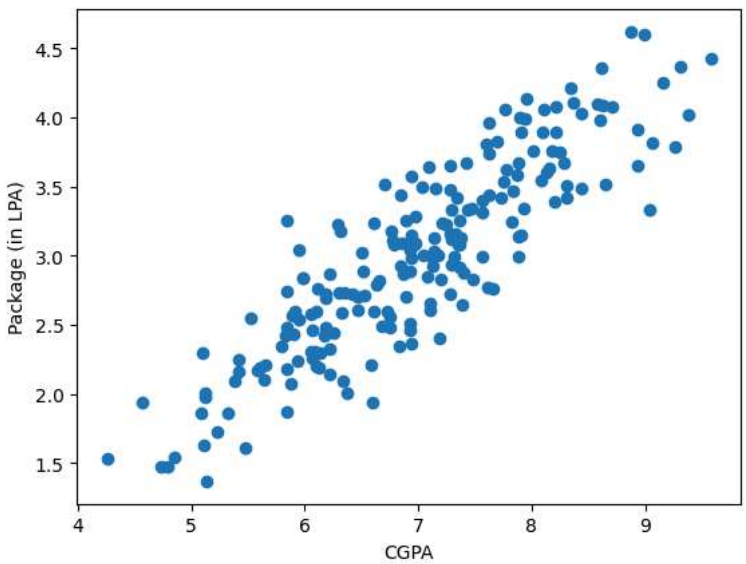
```
df.head()
```

	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57

Next steps: Generate code with df New interactive sheet

```
plt.scatter(df['cgpa'], df['package'])
plt.xlabel('CGPA')
plt.ylabel('Package (in LPA)')
```

```
Text(0, 0.5, 'Package (in LPA)')
```




```
# Selects all rows, start at column 0 and stop before column 1
X = df.iloc[:,0:1] # So it selects only column 0
# Selects all rows Selects the last column This becomes the output/target (y)
y = df.iloc[:,-1]
```

```
X
```

	cgpa
0	6.89
1	5.12
2	7.82
3	7.42
4	6.94
...	...
195	6.93
196	5.89
197	7.21
198	7.63
199	6.22

200 rows × 1 columns

Next steps: Generate code with X New interactive sheet

```
y
```

	package
0	3.26
1	1.98
2	3.25
3	3.67
4	3.57
...	...
195	2.46
196	2.57
197	3.24
198	3.96
199	2.33

200 rows × 1 columns

dtype: float64

```
# train test split
from sklearn.model_selection import train_test_split
# random_state is used to control randomness so results stay the same every time we run the code.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=2)
```

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
```

```
lr.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
X_test
```

	cgpa
112	8.58
29	7.15
182	5.88
199	6.22
193	4.57
85	4.79
10	5.32
54	6.86
115	8.35
35	6.87
12	8.94
92	7.90
13	6.93
126	5.91
174	7.32
2	7.82
44	5.09
3	7.42
113	6.94
14	7.73
23	6.19
25	7.28
6	6.73
134	7.20
165	8.21
173	6.75
45	7.87
65	7.60
48	8.63
122	5.12
178	8.15
64	7.36
9	8.31
57	6.60
78	6.59
71	7.47
128	7.93
176	6.29
131	6.37
53	6.47

Next steps: Generate code with `X_test` New interactive sheet

```
y_test
```

	package
112	4.10
29	3.49
182	2.08
199	2.33
193	1.94
85	1.48
10	1.86
54	3.09
115	4.21
35	2.87
12	3.65
92	4.00
13	2.89
126	2.60
174	2.99
2	3.25
44	1.86
3	3.67
113	2.37
14	3.42
23	2.48
25	3.65
6	2.60
134	2.83
165	4.08
173	2.56
45	3.58
65	3.81
48	4.09
122	2.01
178	3.63
64	2.92
9	3.51
57	1.94
78	2.21
71	3.34
128	3.34
176	3.23
131	2.01
53	2.61

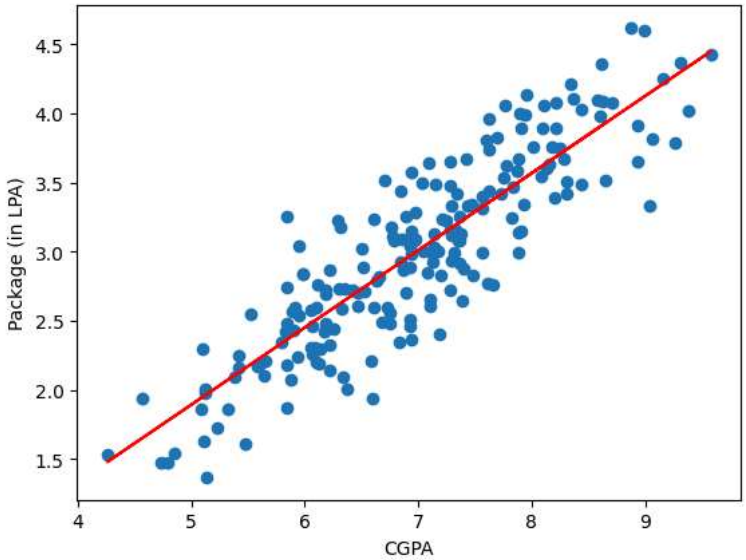
dtype: float64

```
lr.predict(X_test.iloc[0:1])
```

```
array([3.89111601])
```

```
plt.scatter(df['cgpa'], df['package'])
plt.plot(X_train, lr.predict(X_train), color = 'red')
plt.xlabel('CGPA')
plt.ylabel('Package (in LPA)')
```

```
Text(0, 0.5, 'Package (in LPA)')
```




```
# gives the slope (weight) of the feature in the
# trained Linear Regression model, showing how much
# output changes when input increases by 1 unit.
m = lr.coef_
```

```
# gives the constant value (c) in Linear Regression,
# i.e., the predicted output when all input features are 0.
b = lr.intercept_
```

```
# y = mx + b

m * 8.5 + b
```

```
array([3.84647985])
```

```
m * 100 + b
```

```
array([54.89908542])
```

```
print("Equation: y = ", m[0], "x +", b)
```

```
Equation: y =  0.557951973425072 x + -0.8961119222429144
```

```
y_pred = lr.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

print(mean_absolute_error(y_test, y_pred))
print(mean_squared_error(y_test, y_pred))
```